

HBnB Technical Architecture & Design Document



Karen Andrianaharison, Max Brook , Pavith Raj & Joleen Thelen

Table of contents

1.0 Introduction.....	3
2.0 High-Level Architecture.....	3
2.1 Presentation layer.....	5
2.2 Facade calls.....	5
2.3 Business Logic Layer.....	5
2.4 Persistence Layer.....	6
Example of a request flow: User Review submission.....	6
3.0 Business Logic Layer – Class Diagram.....	7
3.1 Purpose of This Diagram.....	7
3.2 Core Entities.....	8
3.2.1 User.....	8
3.2.2 Property.....	8
3.2.3 Amenity.....	9
3.2.4 Review.....	9
4.0 API Interaction Flow – Sequence Diagrams.....	10
4.1 User creation.....	10
4.2 New place creation.....	12
4.3 Review creation.....	14
4.4 Listing places.....	16

1.0 Introduction

This document provides a comprehensive technical blueprint for the HBnB project. It consolidates architectural diagrams and explanatory notes into a document that will guide the implementation of the system. The document captures the system's high-level architecture, business logic design, and API interaction flows.

HBnB is an application designed using a layered architecture to ensure maintainability, scalability, and separation of concerns. This document is intended for developers, reviewers, and stakeholders who require a clear understanding of how the system components interact and how the overall design supports the project's objectives.

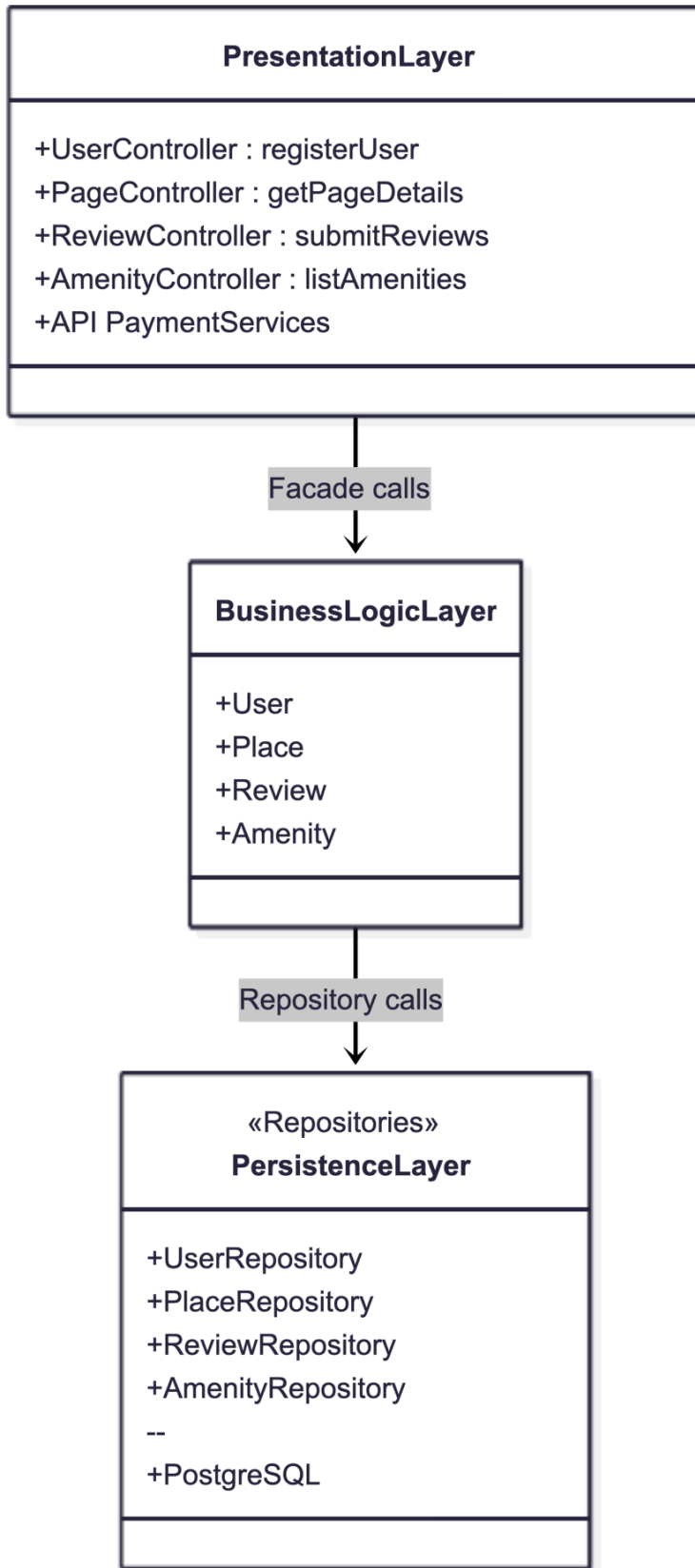
2.0 High-Level Architecture

The High-Level Package Diagram illustrates the overall structure of the HBnB application and how requests flow through the system in itself and how responsibilities are segmented among them. The layered architecture separates the concerns while the facade pattern simplifies the communication between the business logic and the API.

This diagram illustrates how the system is organized and how each layer interacts with each other.

The system follows a layered architecture,

- **Presentation Layer (API):** User interactions and HTTP requests.
- **Business Logic Layer:** Entities, rules, and logic for application.
- **Persistence Layer:** Data storage and retrieval.
- **Facade Layer:** Serves as an entry point that coordinates communication between layers.



2.1 Presentation layer

These are the various controllers of the presentation layer,

- + UserController : registerUser
- + PageController : getPageDetails
- + ReviewController : submitReviews
- + AmenityController : listAmenities
- + API PaymentServices

The four main functions of this layer is as follows,

- Receiving HTTP requests from browser / mobile app
- Input validation
- Calling the facade in the Business Logic layer (see section 3.0)
- Returning of JSON responses

2.2 Facade calls

The main purpose of this is to centralise the business rules and to keep the controllers simple.

2.3 Business Logic Layer

The entirety of the business logic resides in this layer. Certain rules are strictly enforced here (a user who stays in the property can only review for example) while the relationships between the entities are managed as well. Furthermore, various calls for repositories to save and fetch data is handled here while validation and logic is also applied

For example, When a user submits a review,

1. A Check to see **if** property **exists**
2. A Check to see **if** the user **booked** property
3. **Allow** the Creation of the review
4. **Save** the review using ReviewRepository

2.4 Persistence Layer

This layer is responsible for the access of data. Data is saved , updated , deleted and fetched.

Example of a request flow: User Review submission

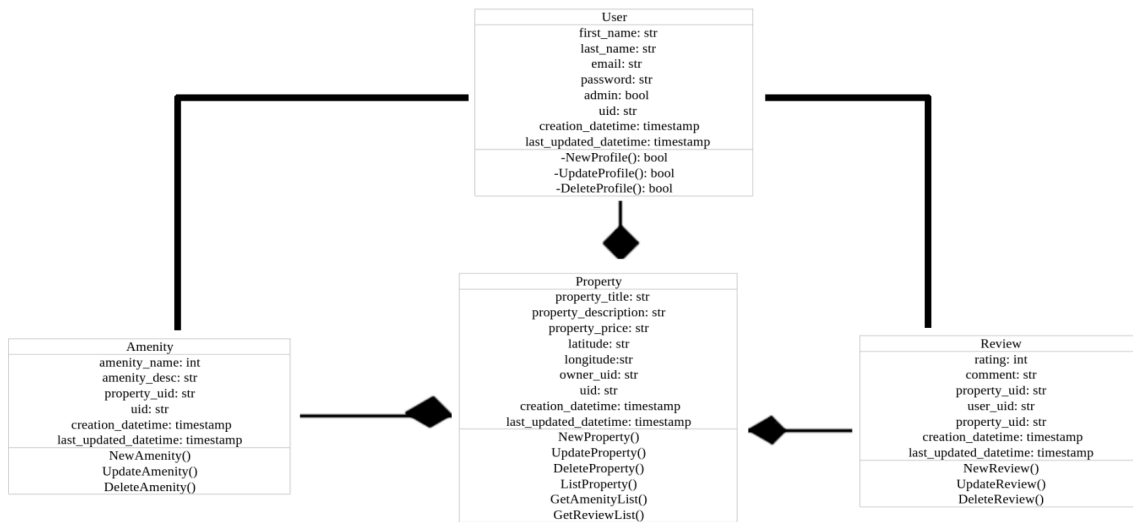
1. `ReviewController.submitReview()` receives the HTTP request
2. Calls `facade.add_review(data)`
3. Facade calls `Review` business class
4. Business class checks the rules
5. Calls `ReviewRepository.save(review)`
6. Repository communicates with PostgreSQL
7. Response is returned.

3.0 Business Logic Layer – Class Diagram

3.1 Purpose of This Diagram

This class diagram represents the core **Business Logic layer** of the HBnB application. It defines the main domain entities, their attributes, behaviors, and relationships. This layer encapsulates all business rules and is in between the **API layer** and the **persistence layer**.

This class diagram fits within a layered architecture using a facade pattern, where the Business Logic layer exposes controlled operations to the API layer. All validation, entity rules, and interactions are handled there.



User > Property: A user can own multiple properties.

Property > Amenity: Amenities are destroyed if the property is deleted.

Property > Review: Reviews belong to a specific property.

User > Review: Users can author multiple reviews.

3.2 Core Entities

3.2.1 User

The **User** class represents all registered users of the platform, including users and admins.

Attributes

- **first_name, last_name**: Personal identification details
- **email**: Unique identifier for authentication
- **password**: Hashed user password
- **admin**: Boolean flag indicating administrative privileges
- **uid**: Unique system identifier
- **creation_datetime, last_updated_datetime**: Audit timestamps

Methods

- **NewProfile()** – Creates a new user profile
- **UpdateProfile()** – Updates existing user details
- **DeleteProfile()** – Removes a user profile

3.2.2 Property

The **Property** class represents accommodation listings created by users.

Attributes

- **property_title, property_description**: Descriptive metadata
- **property_price**: Cost per stay
- **latitude, longitude**: Geolocation data
- **owner_uid**: References the owning user
- **uid**: Unique property identifier
- **creation_datetime, last_updated_datetime**: Audit timestamps

Methods

- **NewProperty()** – Creates a new property listing
- **UpdateProperty()** – Modifies property details
- **DeleteProperty()** – Removes a property
- **ListProperty()** – Retrieves available properties
- **GetAmenityList()** – Fetches amenities linked to the property
- **GetReviewList()** – Fetches reviews linked to the property

3.2.3 Amenity

The **Amenity** class defines individual amenities associated with a property (e.g., Wi-Fi, Pool).

Attributes

- `amenity_name, amenity_desc`: Amenity metadata
- `property_uid`: Foreign key reference to Property
- `uid`: Unique amenity identifier
- `creation_datetime, last_updated_datetime`: Audit timestamps

Methods

- `NewAmenity()` – Adds a new amenity to a property
- `UpdateAmenity()` – Updates amenity details
- `DeleteAmenity()` – Removes an amenity

3.2.4 Review

The **Review** class captures user feedback and ratings for properties.

Attributes

- `rating`: Numerical evaluation score
- `comment`: Textual feedback
- `property_uid`: Associated property
- `user_uid`: Authoring user
- `creation_datetime, last_updated_datetime`: Audit timestamps

Methods

- `NewReview()` – Creates a new review
- `UpdateReview()` – Modifies an existing review
- `DeleteReview()` – Removes a review

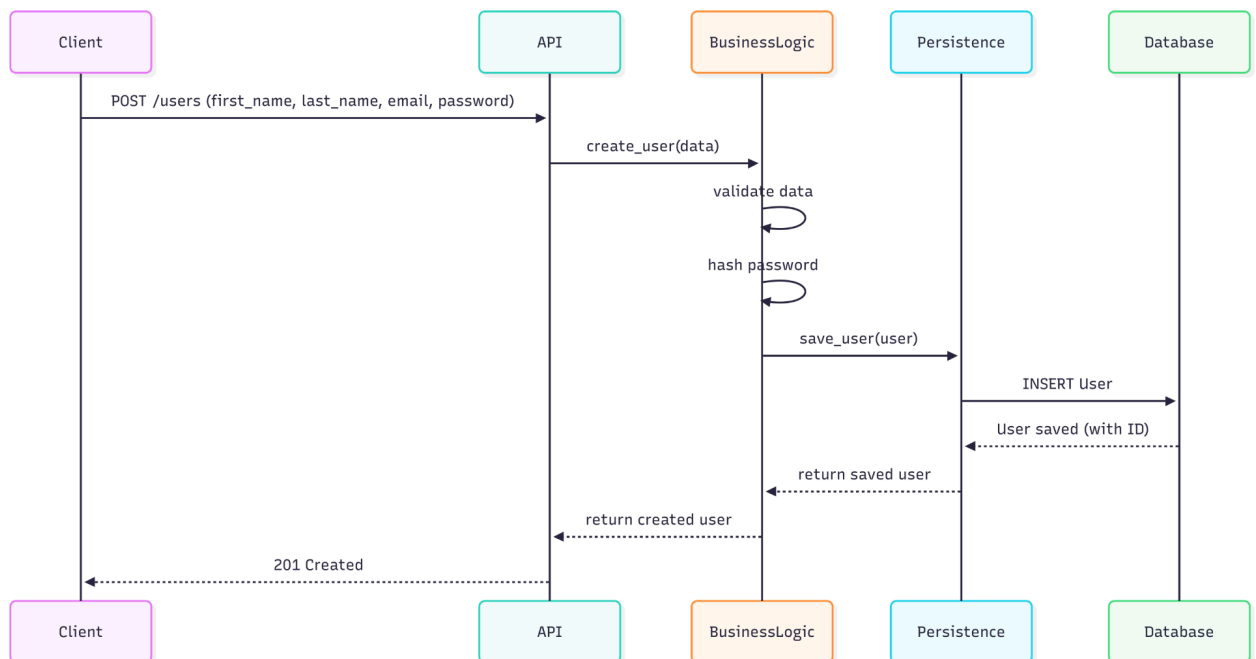
4.0 API Interaction Flow – Sequence Diagrams

The four Sequence Diagrams demonstrate how different components communicate during API calls. Flows such as creating a new user , creating a new place , allowing the user to post a review (provided the user stayed) , and fetching places with amenities.

General Interaction Flow:

1. Client sends HTTP request to API.
2. API forwards request to the Facade.
3. Facade invokes appropriate Business Logic methods.
4. Business Logic interacts with Persistence Layer.
5. Response is returned back through the Facade to the API and then to the client.

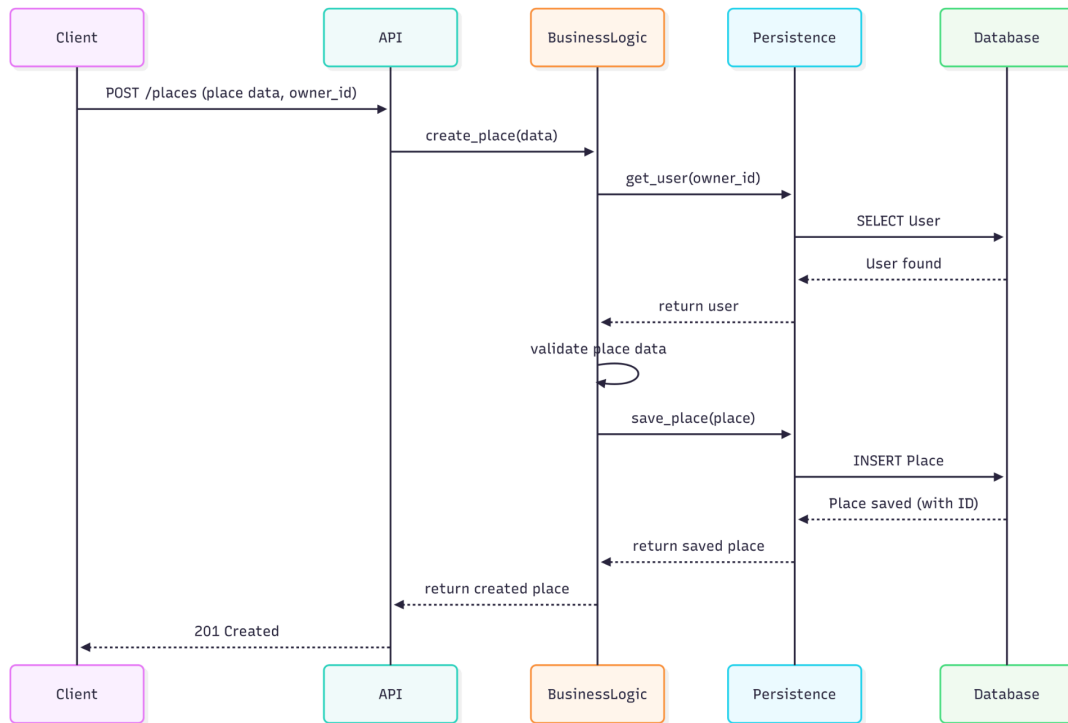
4.1 User creation



Interaction Flow: (Create New user)

1. Client (the browser) sends HTTP requests to API through POST. `POST /users (first_name, last_name, email, password)`
2. API forwards requests to the Facade. `create_user(data)`
3. Facade invokes appropriate Business Logic methods. Such as email format checks, password length checks, required fields exist checks.
4. Business Logic interacts with Persistence Layer ; business layer calls the repository. `save_user(user)`
5. Repository performs `INSERT User` and Database generates an ID. Returns the saved user.
6. Response is returned back through the Facade to the API and then to the client. `201 Created`

4.2 New place creation

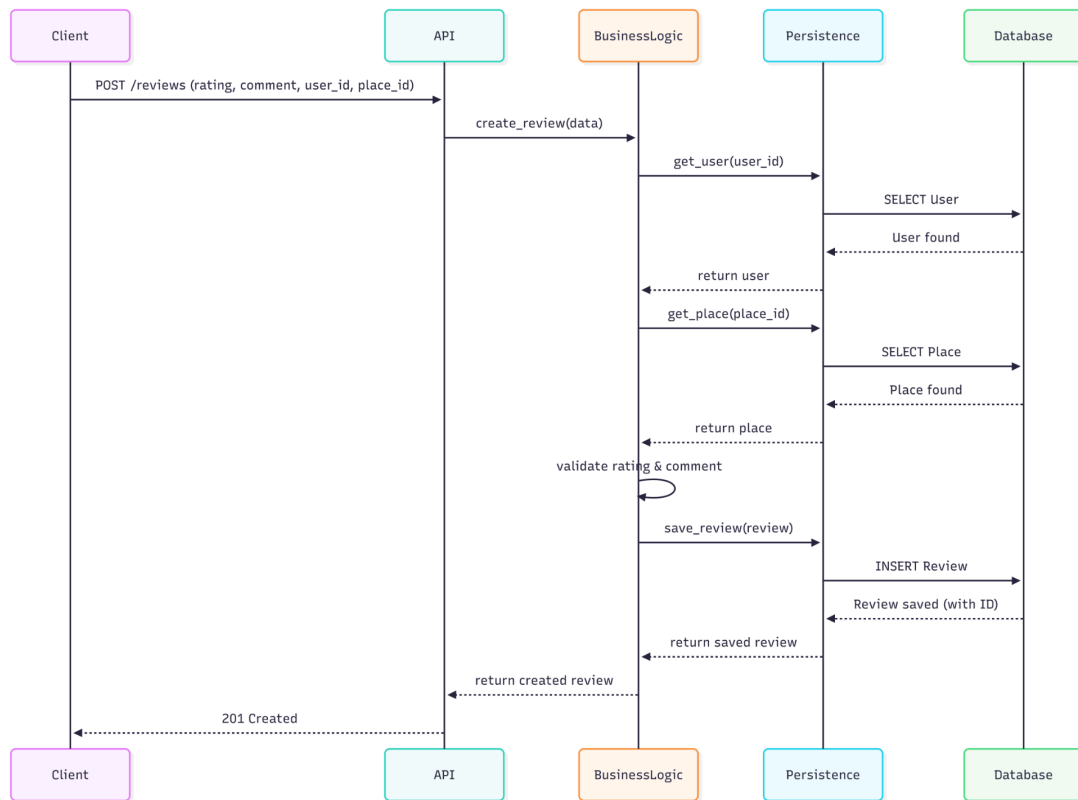


Interaction Flow (Create Place / Property)

1. Client (the browser) sends an HTTP request to the API using POST.
`POST /places (place_data, owner_id)`
2. API forwards the request to the Facade.
`create_place(data)`
3. Facade invokes the appropriate Business Logic methods.
 - Verify that the `owner_id` exists by calling: `get_user(owner_id)`
 - Validate place data (title, description, price, coordinates, required fields)

4. Business Logic interacts with the Persistence Layer.
 - First retrieves the owner from the repository:
`UserRepository.get(owner_id)`
 - Then calls the place repository to save the place: `save_place(place)`
5. Repository performs `INSERT Places` and the Database generates an ID.
 - Database returns the saved place (with generated ID)
6. Response is returned back through the Facade to the API and then to the client.
`201 Created` with the created place details.

4.3 Review creation

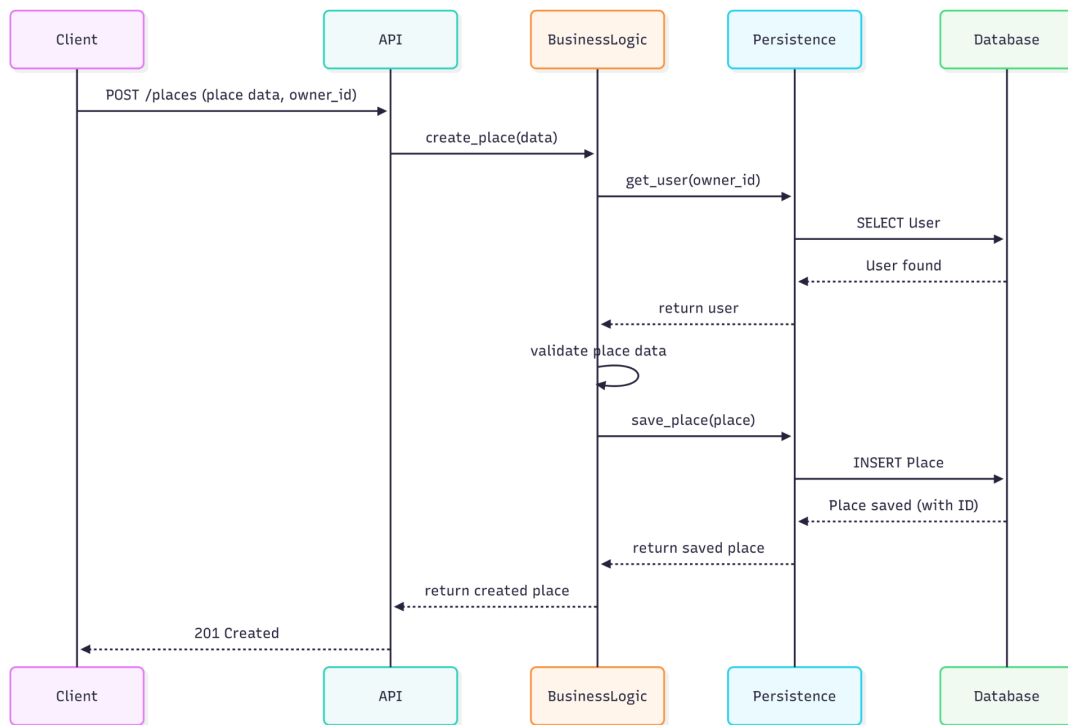


Interaction Flow (Review Creation)

1. Client (the browser) sends an HTTP request to the API using POST.
`POST /reviews (rating, comment, user_id, place_id)`
2. API forwards the request to the Facade.
`create_review(data)`
3. Facade invokes the appropriate Business Logic methods.
 - Verify that the `user_id` exists: `get_user(user_id)`
 - Verify that the `place_id` exists: `get_place(place_id)`
 - Validate rating range and comment content (required fields, limits)

4. Business Logic interacts with the Persistence Layer.
 - Retrieve user from repository: `UserRepository.get(user_id)`
 - Retrieve place from repository: `PlaceRepository.get(place_id)`
 - Save the review: `save_review(review)`
5. Repository performs `INSERT Review` and the Database generates an ID.
 - Database returns the saved review (with generated ID)
6. Response is returned back through the Facade to the API and then to the client.
`201 Created` with the created review details.

4.4 Listing places



Interaction Flow (List Places with Criteria)

1. Client (the browser / frontend / Postman) sends an HTTP request to the API using GET.
2. API forwards the request to the Facade `list_places(criteria)`
3. Facade invokes the appropriate Business Logic methods and business Logic interacts with the Persistence Layer.
4. Call repository method with filters: `PlaceRepository.get_places(criteria)`
5. Repository builds the query and interacts with the Database. First Construct query based on filters and then `SELECT * FROM places WHERE criteria`
6. Database returns the list of matching places.
7. Repository returns the list of places to the Business Logic.
8. Business Logic returns the filtered list to the Facade.

9. Response is returned back through the Facade to the API and then to the client.

200 OK with JSON list of places